
DUNE

Arrakis Dominion Distributed

Memoria

Programación en Entornos Distribuidos

Vic, Fer y Karin

github.com/VicM14/dune-arrakis

1. Análisis de la aplicación

Hicimos un simulador de gestión por turnos inspirado en el universo de Dune. El jugador gestiona una Casa Menor que opera enclaves de criaturas: compra suministros, construye instalaciones, alimenta y traslada animales, recibe visitantes y gana dinero (solaris), avanzando mes a mes.

La parte técnica del proyecto es que lo dividimos en varios programas independientes que se comunican entre sí por red, en lugar de hacerlo todo en uno solo. La solución tiene seis partes:

- ▶ **Dune.Domain** — define todas las entidades del juego (criaturas, enclaves, instalaciones...). Lo usan todos los demás componentes.
- ▶ **Dune.SimulationService** — el servidor principal. Ejecuta las rondas mensuales y coordina todo.
- ▶ **Dune.PersistenceService** — guarda y carga las partidas en un archivo JSON.
- ▶ **Dune.AdminClient** — cliente de consola para manejar el juego por texto.
- ▶ **Dune.Unity** — la interfaz gráfica en Unity.
- ▶ **Dune.Tests** — pruebas automáticas para comprobar que las fórmulas del juego dan los resultados correctos.

2. Fundamentos distribuidos aplicados

En esta sección relacionamos los conceptos teóricos de la asignatura con lo que realmente implementamos:

Concepto	Cómo lo aplicamos
Sistema distribuido	Dos clientes (consola y Unity) y dos servidores funcionan por separado pero el jugador lo ve como una sola aplicación.
Cliente-servidor	Los clientes mandan peticiones al SimulationService y esperan respuesta antes de continuar.
Eventos internos	Al terminar cada mes publicamos un evento y varios módulos reaccionan por separado sin que nadie tenga que llamarlos directamente.
Consistencia eventual	El guardado de la partida ocurre después de responder al cliente, no al mismo tiempo. El juego sigue funcionando aunque el guardado tarde un poco.
Tolerancia a fallos	El SimulationService arranca aunque el servicio de guardado no esté activo, e intenta guardar de nuevo más tarde si falla.
Responsabilidad separada	Los clientes nunca tocan el archivo de guardado directamente. Solo el PersistenceService sabe dónde está.

3. Diseño de arquitectura

Los programas se comunican por HTTP mandándose datos en formato JSON. Separamos claramente qué hace cada parte: el dominio define las reglas, el SimulationService las ejecuta, el PersistenceService guarda, y los clientes (consola y Unity) muestran y reciben órdenes.

Tanto la consola como Unity hablan únicamente con el SimulationService en el puerto 5000. Ese servicio es el único que puede pedirle al PersistenceService (puerto 5032) que guarde o cargue algo. Los clientes no saben nada de cómo ni dónde se guardan los datos.

◆ 3.1. La interfaz gráfica en Unity

Construimos la interfaz en Unity 2021.3 LTS con estética pixel art inspirada en juegos retro como UNDUNE II. Unity actúa como un cliente HTTP más, exactamente igual que la consola pero con pantalla visual.

Tiene tres pantallas:

- ▶ **Menú principal:** con botones para nueva partida y cargar partida guardada
- ▶ **Selección de escenario:** para escribir el nombre del jugador y elegir entre Arrakeen, Giedi Prime y Caladan
- ▶ **Pantalla de juego:** la pantalla principal donde ocurre todo
- ▶

La pantalla de juego tiene esta organización:

- ▶ **Barra superior:** nombre del jugador, mes actual y solaris
- ▶ **Panel izquierdo:** lista de enclaves e instalaciones
- ▶ **Centro:** grid visual con las instalaciones representadas como tiles
- ▶ **Panel derecho:** detalle de la instalación o criatura seleccionada, con las criaturas ordenadas de mayor a menor salud
- ▶ **Panel inferior:** historial de eventos y botones de acción (construir, comprar, mover, trasladar, descartar, simular mes)

◆ 3.2. Cómo se mandan las peticiones desde Unity

Cada acción del jugador lanza una petición HTTP al SimulationService y espera respuesta. Usamos corrutinas de Unity para no bloquear la pantalla mientras esperamos. El GameManager es el encargado de guardar el estado actual de la partida y avisar a la interfaz cuando hay cambios.

◆ 3.3. Tratamiento de errores

Si el servidor de guardado no responde, el SimulationService lo reintenta hasta tres veces esperando cada vez más tiempo antes del siguiente intento. Si sigue fallando, el juego sigue funcionando en memoria sin perder el estado.

En Unity, si una petición falla, se muestra un mensaje de error al jugador en pantalla. Nunca se deja la interfaz bloqueada sin feedback.

4. Concurrency y sincronización

El `SimulationService` puede recibir varias peticiones a la vez, así que tuvimos que asegurarnos de que dos operaciones no modifiquen la partida al mismo tiempo.

Lo resolvimos con un candado (`SemaphoreSlim`): cuando llega una petición que va a cambiar algo, coge el candado, hace su trabajo, y lo suelta. Si llega otra petición mientras tanto, espera su turno. Así nunca se pisan dos operaciones entre sí.

Además, cuando se cierra un mes, los distintos módulos que tienen que reaccionar (guardar la partida, registrar el evento en el log...) lo hacen todos a la vez en paralelo en lugar de uno detrás del otro, lo que hace que la ronda tarde menos en completarse.

5. Persistencia y consistencia

◆ 5.1. Cómo guardamos la partida

Guardamos todo en un archivo JSON con formato legible, de forma que si hay que revisar los datos se puede abrir directamente con cualquier editor.

El punto más difícil fue guardar y cargar correctamente las criaturas. Como hay cinco tipos distintos (Gusano de Arena, Tigre Laza, etc.) que heredan de una clase común, el archivo JSON necesita saber cuál es cada una al cargar. Usamos unos atributos especiales de .NET (`[JsonPolymorphic]` y `[JsonDerivedType]`) que meten automáticamente un indicador del tipo en el JSON. Sin esto, al cargar la partida las criaturas perdían su comportamiento específico.

◆ 5.2. Cuándo se guarda

La partida se guarda automáticamente al terminar cada ronda mensual, como efecto secundario del evento de fin de mes. También se puede guardar manualmente desde la interfaz en cualquier momento. Al arrancar el programa, intenta recuperar la última partida guardada; si no hay ninguna o el servicio no está disponible, empieza con estado vacío sin dar error.

◆ 5.3. Consistencia eventual

El guardado ocurre después de que el servidor responde al cliente, no antes. Esto significa que hay un momento muy breve en que el juego ha avanzado pero el archivo todavía no está actualizado. Lo aceptamos conscientemente a cambio de que el juego sea más rápido y el guardado no bloquee la respuesta al jugador.

6. Seguridad

Al ser un prototipo que corre en local, la seguridad no era una prioridad, pero sí pusimos algunos controles básicos:

- ▶ Todas las operaciones comprueban que los datos son válidos antes de ejecutarse: que hay fondos suficientes, que la instalación tiene espacio, que la criatura cumple los requisitos de traslado, etc.

-
- ▶ Si algo no es válido, el servidor responde con un mensaje de error claro en lugar de ejecutar una operación incorrecta.
 - ▶ Los clientes no pueden acceder al archivo de guardado directamente; todo pasa por los servicios.

7. Ordenación y búsqueda

Para cumplir con los requisitos del enunciado implementamos estas ordenaciones y búsquedas:

- ▶ **Criaturas por salud.** Tanto en el backend como en Unity, las criaturas se muestran de mayor a menor salud, para que el jugador vea de un vistazo cuáles están en riesgo.
- ▶ **Eventos cronológicos.** El historial de eventos mantiene el orden en que se fueron añadiendo, así que siempre se ve en orden cronológico sin necesidad de reordenar.
- ▶ **Búsqueda por ID.** Cuando hay que localizar un enclave, instalación o criatura concretos, buscamos en la lista por su identificador único. Con la cantidad de elementos del juego (decenas como máximo) esto es suficientemente rápido.
- ▶ **Cálculos con LINQ.** Usamos las funciones de consulta de C# para calcular cosas como la salud media de las criaturas de una instalación o el total de hectáreas ocupadas.

8. Errores y dificultades que encontramos

◆ 8.1. Scripts iniciales incorrectos

Al empezar la parte de Unity ya teníamos unos scripts creados, pero tenían varios errores: los modelos de datos no coincidían con lo que devuelve el servidor (campos que no existían, tipos incorrectos) y el código intentaba conectarse directamente al servicio de guardado en lugar de pasar por el SimulationService. Tuvimos que reescribir todo desde cero una vez identificamos los problemas.

◆ 8.2. El JSON de las criaturas no cargaba bien

Las criaturas llegaban vacías desde el servidor. Tardamos un rato en entender que el problema era que el JSON del backend incluye un campo especial (\$type) para distinguir el tipo de criatura, y el sistema de lectura de JSON que usa Unity por defecto no lo entiende. La solución fue instalar una librería adicional (Newtonsoft.Json) que sí lo soporta.

◆ 8.3. Los botones no respondían en una pantalla

En la pantalla de selección de escenario, los botones no hacían nada al pulsarlos y el campo de texto no dejaba escribir. El motivo era que faltaba el objeto EventSystem en esa escena, que es el que gestiona toda la interacción con la interfaz en Unity. Al añadirlo se solucionó.

◆ 8.4. Las tarjetas de criaturas no se mostraban bien

El panel lateral que muestra la lista de criaturas tardó varios intentos en funcionar correctamente. El problema era que el área de scroll era demasiado grande y tapaba otros elementos del panel, y además el contenido no crecía al añadir tarjetas porque le faltaba una configuración específica (el Content Size Fitter).

9. Conclusiones

El proyecto funciona como juego y cumple los requisitos técnicos de la asignatura: tiene varios servicios separados que se comunican por red, maneja bien los fallos parciales, guarda y carga partidas, y tiene una interfaz gráfica funcional.

Lo que más aprendimos fue a pensar en que la red puede fallar en cualquier momento y que dos partes del sistema pueden ir a ritmos distintos. Antes de esta práctica escribíamos código asumiendo que todo funciona siempre, y ahora entendemos por qué hay que plantearse qué pasa si algo falla a medias.

La parte de Unity fue la más difícil para nosotros porque ninguno lo había usado en este contexto. Empezamos con código incorrecto, tuvimos que entender cómo funciona la comunicación HTTP dentro de Unity, y coordinar cambios entre los tres sin rompernos el trabajo fue un aprendizaje extra en sí mismo.

El resultado es una interfaz jugable con estética sencilla y todas las acciones del juego operativas desde la pantalla.